ИТМО

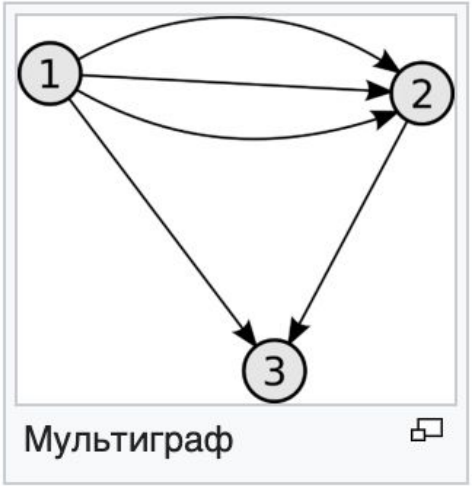
Алгоритмы и структуры данных

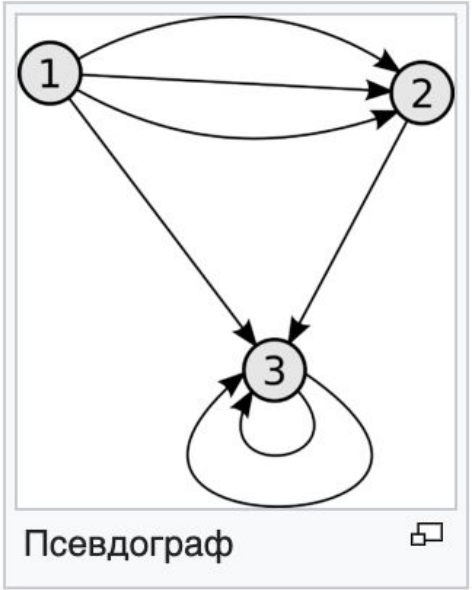
Лекция 2

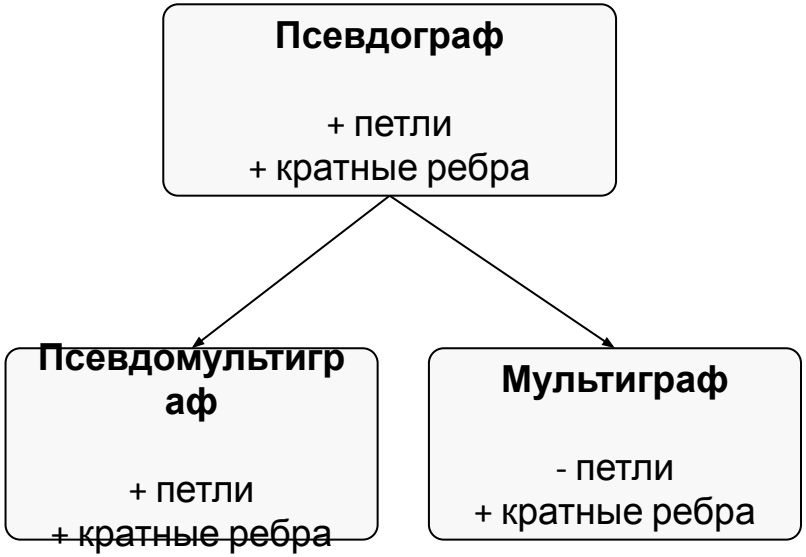
Алгоритмы на графах

Что успеем

О псевдо и мульти графах



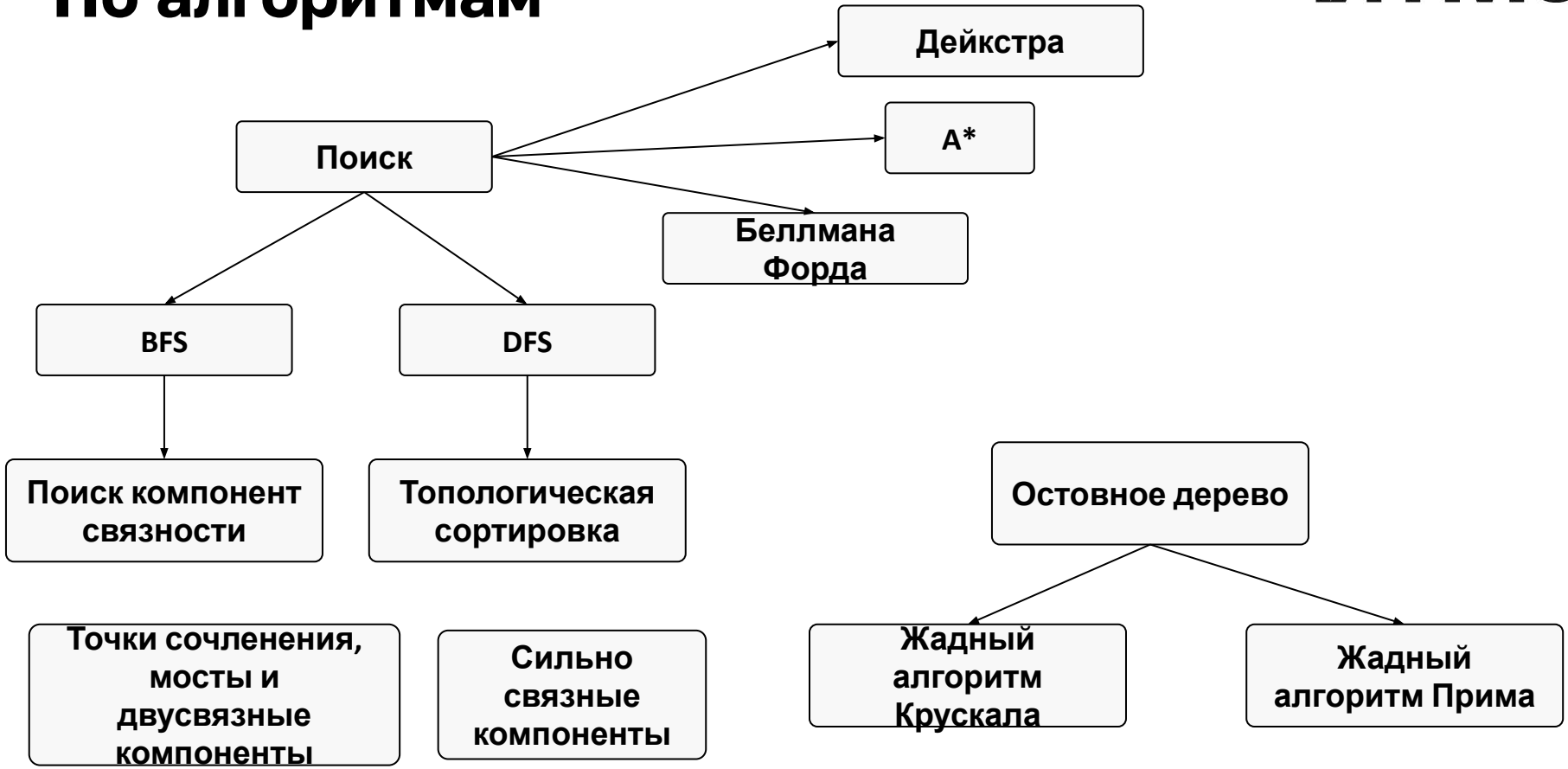




Говорят, что названием «псевдомультиграфом» подчеркивают наличие нескольких кратных ребер

Ссылка на вики ИТМО

По алгоритмам

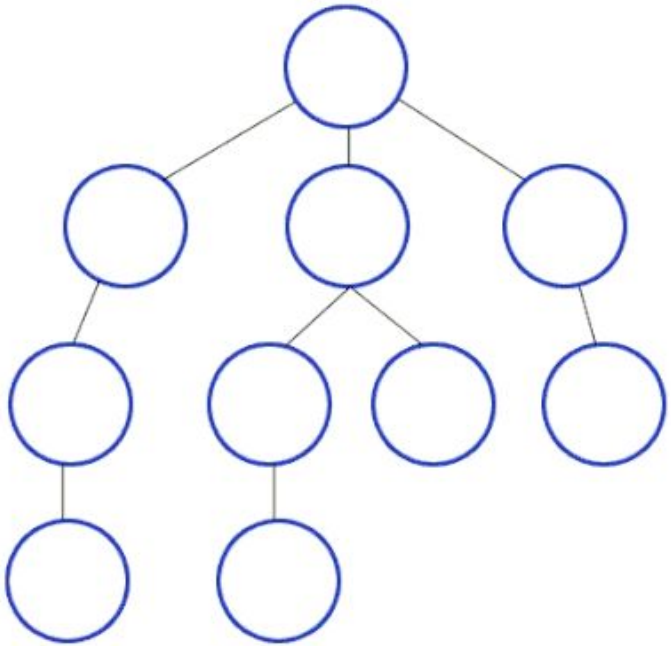


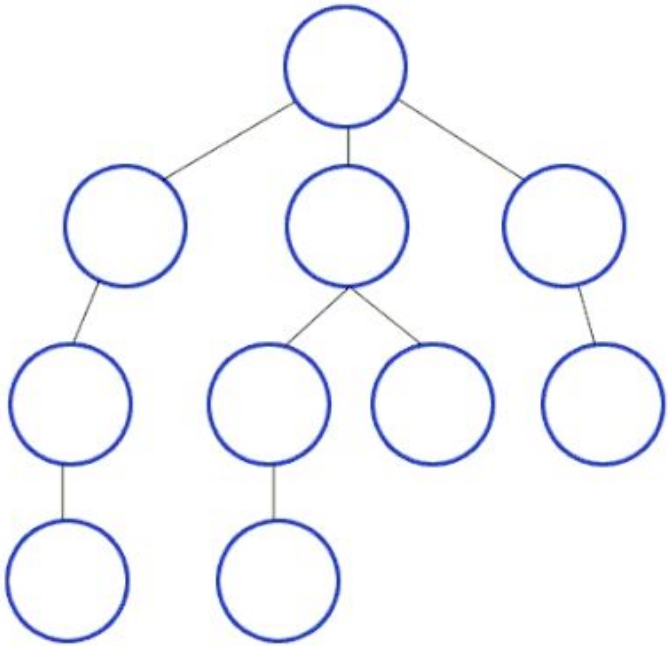
Имеющийся граф (схематично)

ИТМО

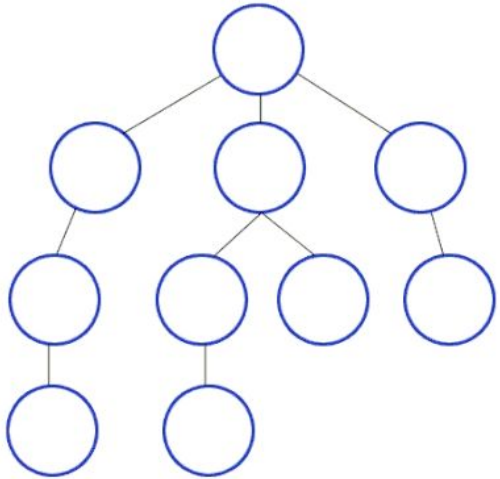
```
// Определяем структуру для узла графа
typedef struct {
  int node_id; // Данные узла (может быть ID)
  Vector* neighbors;
} Node;

typedef struct {
   Vector* nodes; // Вектор указателей на узлы графа
} Graph;
```

Breadth-first and Depth-first search





Поиск в ширину

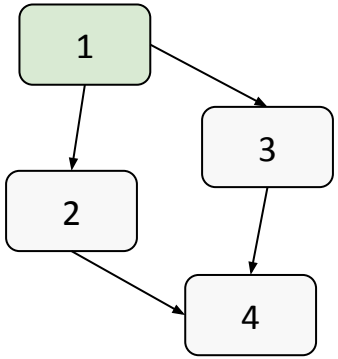


1. Структура?
2. Подход?

Код для поиска в ширину

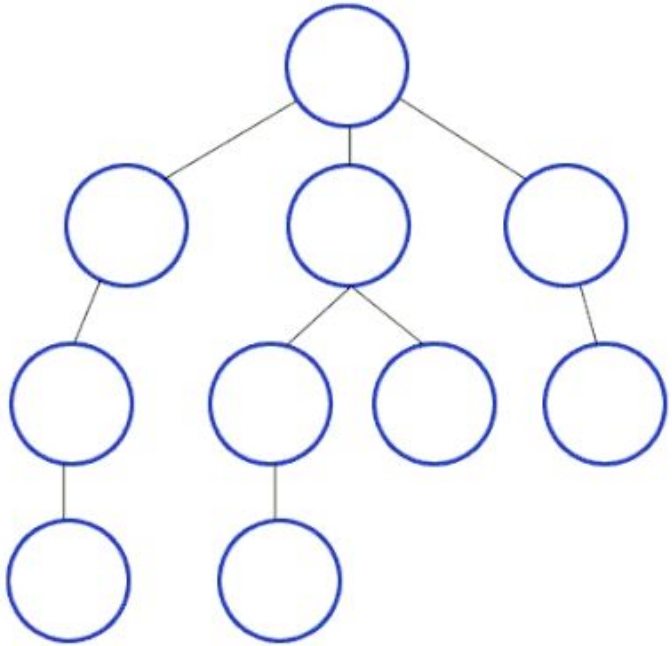
ИТМО

```
void bfs(Graph* graph, unsigned int node_id_start) {
    GenericList* list = createList(sizeof(unsigned int));
    appendItem(list, &node_id_start);

    while(listLength(list) > 0) {
        unsigned int node_next_id = *(unsigned int*)popItem(list, 0);
        GNode* node_next = findNode(graph, node_next_id);

        for (int i = 0; i < node_next->siblings->size; i++) {
            GNode *node = getVectorItem(node_next->siblings, i);
            appendItem(list, &node->node_id);
        }
    }
}
```



cout:
Visited node: 1
Visited node: 2
Visited node: 3
Visited node: 4
Visited node: 4

Поиск в глубину

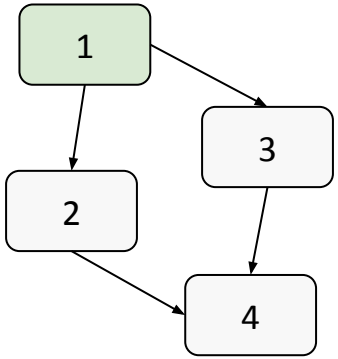


1. Структура?
2. Подход?

Поиск в глубину

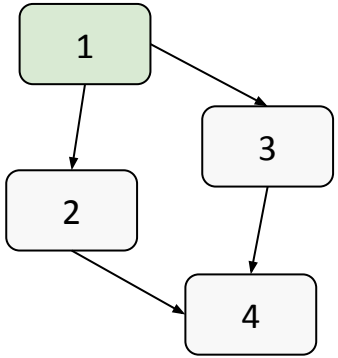
ИТМО

```
void dfs(Graph* graph, unsigned int node_id_start) {
    GNode* node_next = findNode(graph, node_id_start);

    if (node_next->siblings->size == 0) {
        return;
    }

    for (int i = 0; i < node_next->siblings->size; i++) {
        GNode *node = getVectorItem(node_next->siblings, i);
        dfs(graph, node->node_id);
    }
}
```



cout:
Visited node: 1
Visited node: 2
Visited node: 4
Visited node: 3

А теперь сравним



bfs:
Visited node: 1
Visited node: 2
Visited node: 3
Visited node: 4

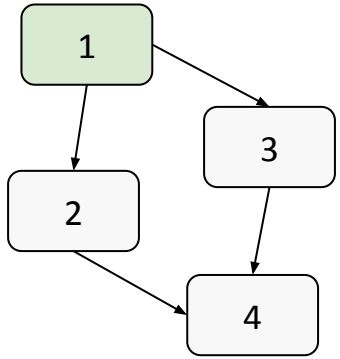
dfs:
Visited node: 1
Visited node: 2
Visited node: 4
Visited node: 3

Какая есть тяга в dfs в текущей реализации?

Уходим от рекурсии через стек

ИТМО

```
void dfs_without_recursion(Node* start) {
   std::stack<Node*> stack; // Стек для обхода
   std::vector<bool> visited(nodes.size(), false);

   stack.push(start);
   while (!stack.empty()) {
       Node* current = stack.top();
       stack.pop();

       // Если текущий узел не был посещен
       if (!visited[current->node_id]) {
           std::cout << "Visited node: " << current->node_id <<
std::endl;
           visited[current->node_id] = true; // Отметить как посещенный

           // Добавить соседей в стек
           for (auto neighbor : current
               stack.push(neighbor);
           }
       }
   }
}
```



dfs:
Visited node: 1
Visited node: 3
Visited node: 4
Visited node: 2

Минутная пауза на осознание

Топологическая сортировка

Топологическая сортировка (Topological sort) — один из основных алгоритмов на графах, который применяется для решения множества более сложных задач.

Ссылка на статью

Будете себя плохо вести, будут вот такие лекции...

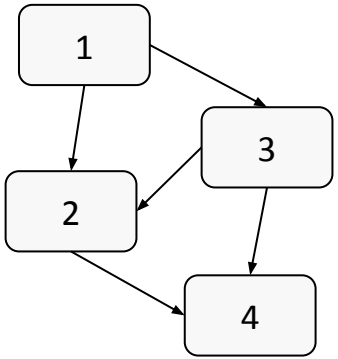
ИТМО

```
const int maxn = 1e5;

bool used[maxn];
vector<int> t;

void dfs(int v) {
    used[v] = true;
    for (int u : g[v])
        if (!used[u])
            dfs(u);
    t.push_back(v);
}

void topological_sort() {
    for (int v = 0; v < n; v++)
        if (!used[v])
            dfs(v);
    reverse(t.begin(), t.end());
}
```

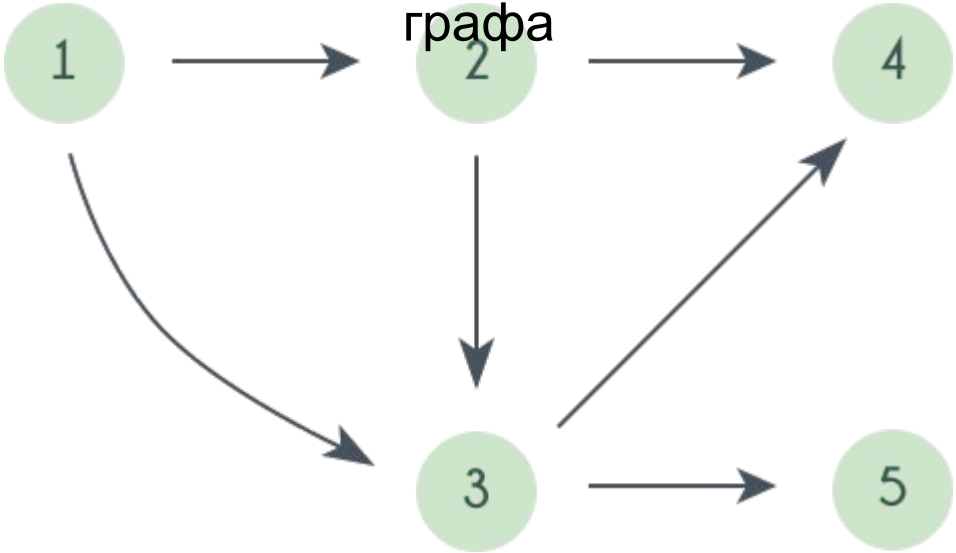
Ссылка на статью

Так вот, топологическая сортировка

Топологическая сортировка - определение порядка обхода вершин в ориентированном ациклическом графе



Пример отсортированного графа



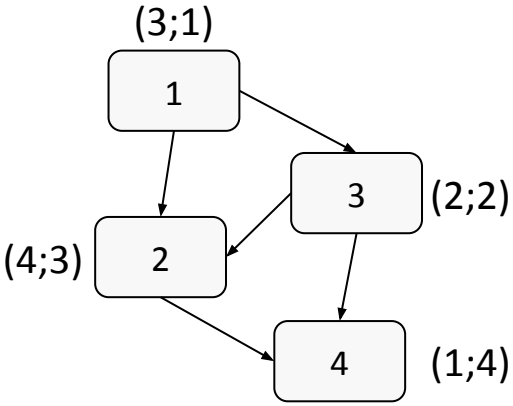
Ссылка на статью (картинку оттуда взял)

Как реализовать топологическую сортировку

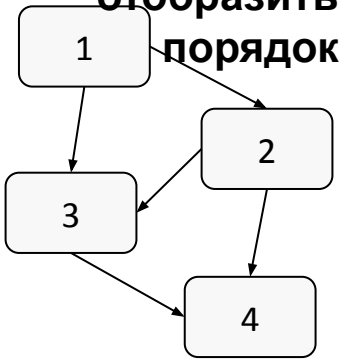
ИТМО

Напомню, как выглядит граф

```
typedef struct {
  int node_id; // Данные узла (может быть ID)
  Vector* neighbors; // Это “из” или “в”?
} Node;

typedef struct {
   Vector* nodes;
} Graph;
```



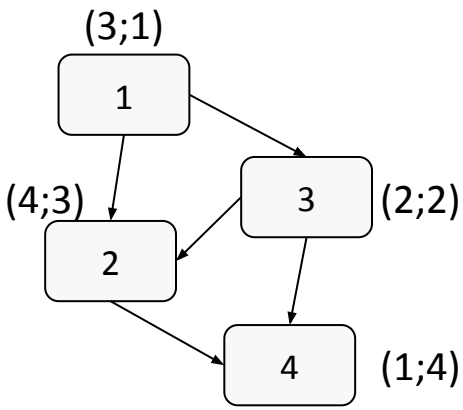
Если отобразить порядок



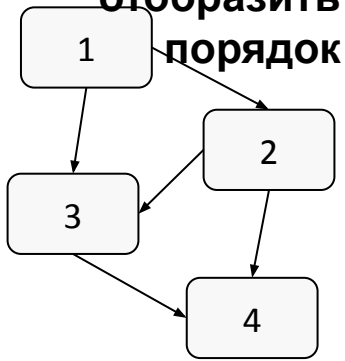
Код топологической сортировки

ИТМО

```
void topologicalSortIter(Graph* graph, unsigned int node_id,
                         HashSetInt* nodes_exclude, GenericList* path) {
    GNode* node = findNode(graph, node_id);
    for (int i = 0; i < node->siblings->size; i++) {
        GNode* node_parent = *(GNode**)getVectorItem(node->siblings, i);
        if (!hashSetIntContains(nodes_exclude, node_parent->node_id)) {
            topologicalSortIter(graph, node_parent->node_id,
                                nodes_exclude, path);
        }
    }

    appendItem(path, &node->node_id);
    hashSetIntAdd(nodes_exclude, &node->node_id);
}
```



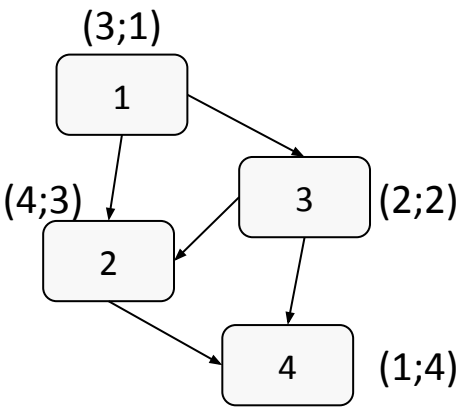
Если отобразить порядок



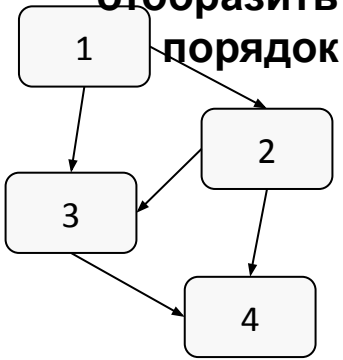
Обёртка

ИТМО

```
void topologicalSort(Graph* graph) {
    GenericList* end_nodes = createList(sizeof(unsigned int));
    for (int i = 0; i < graph->nodes->size; i++) {
        GNode* node = *(GNode**)getVectorItem(graph->nodes, i);
        if (node->siblings->size > 0) {  // Это "B"
            appendItem(end_nodes, &node->node_id);
        }
    }
    GenericList* nodes_exclude = createList(sizeof(unsigned int));
    GenericList* path = createList(sizeof(unsigned int));
    for (int i = 0; i < listLength(end_nodes); i++) {
        unsigned int node_id = popItem(end_nodes, 0);
        topologicalSortIter(graph, node_id, nodes_exclude, path);
    }
}
```



Если отобразить порядок

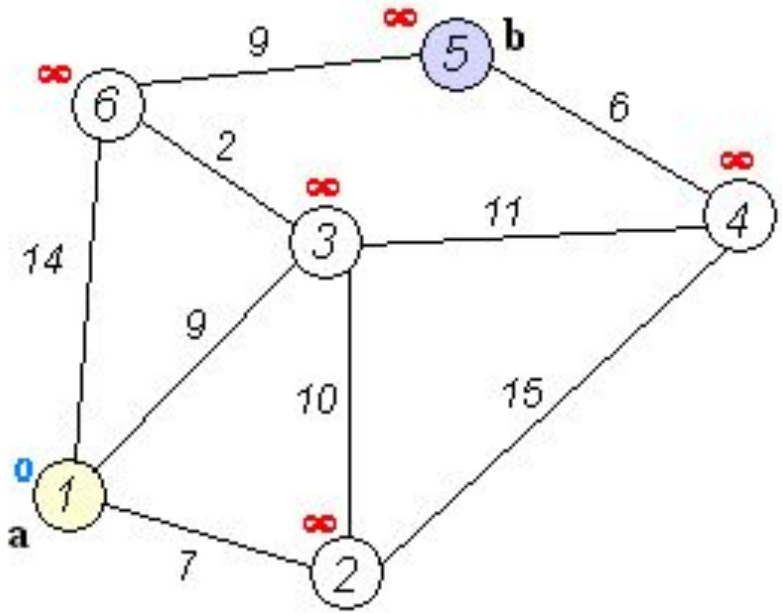


Асимптотика топологической сортировки

Давайте считать вместе)

Минутная пауза на осознание

Дейкстра



одна

гифка

вместо

тысячи

слов

Исходная структура графа

ИТМО

```
typedef struct {
  int node_id;
  Vector* neighbors;
} Node;
```

```
typedef struct {
   Vector* nodes;
} Graph;
```

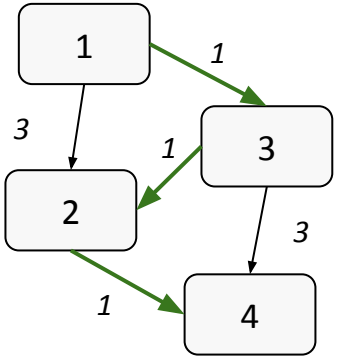
```
void dijkstra(Graph* graph, size_t node_from, size_t node_to) {
   // Создать очередь

   // Пока очередь не пустая и не найден искомый узел
       // Достаем узел с минимальным расстоянием
       // Обновляем расстояние в детях
}
```

Что не сходится?

Алгоритм поиска пути Дейкстра

ИТМО

```
std::vector<Node*> dijkstra(Node* start, Node* end) {
    std::unordered_map<int, int> distances;  // Хранилище для минимальных расстояний
    for (Node* node : nodes) distances[node->node_id] = node->node_id == start->node_id ? 0 : -1;

    std::unordered_map<int, int> previous;  // Хранилище для предыдущих узлов на пути
    std::priority_queue<std::pair<Node*, int>, ...> pq; // Очередь минимальных расстояний приоритетов
    pq.push(std::pair(start, 0));

    while (!pq.empty()) {
        std::pair<Node*, int> current = pq.top(); pq.pop();

        if (current.first == end) {...} // Если достигли конца

        for (std::pair<Node*, int> neighbor : current.first->neighbors) {
            int newDistance = distances[current.first->node_id] + neighbor.second;

            int neigborDistance = distances[neighbor.first->node_id];
            if (neigborDistance == -1 || newDistance < neigborDistance) {
                // Обновить distance, путь в previous и добавить в очередь neighbor
            }
        }
    }

    return {}; // Возвращаем пустой путь, если путь не найден
}
```



Асимптотика в Дейкстре

ИТМО

Реализация очереди	Время работы	Когда выгодно
Массив (линейный поиск)	$O(V^2 + E)$	Плотные графы ($E \approx V^2$)
Список (линейный поиск)	$O(V^2 + E)$	Плотные графы ($E \approx V^2$)
Бинарная куча (min-heap)	$O(E \cdot \log V)$	Разреженные графы ($E \ll V^2$)
Красно-чёрное дерево (RB-tree)	$O(E \cdot \log V)$	Когда нужна упорядоченность или диапазонные операции
Куча Фибоначчи	$O(E + V \cdot \log V)$	Теоретически оптимально, сложно в реализации

Алгоритм поиска пути A*

ИТМО

```
std::vector<Node*> dijkstra(Node* start, Node* end) {
    std::unordered map<int, int> distances;  // Хранилище для минимальных расстояний
    for (Node* node : nodes) distances [node->node_id ] = node->node_id ==  start->node_id ? 0 : -1;

    std::unordered map<int, int> previous;  // Хранилище для предыдущих узлов на пути
    std::priority queue<std::pair<Node*, int>, ...> pq;  // Очередь минимальных расстояний приоритетов
    pq.push(std::pair(start, 0));

    while (!pq.empty()) {
        std::pair<Node*, int> current = pq.top(); pq.pop();

        if (current.first ==  end) {...} // Если достигли конца

        for (std::pair<Node*, int> neighbor : current.first->neighbors) {
            int newDistance = distances [current.first->node_id ] + neighbor.second

            int neigborDistance = distances [neighbor.first->node id ];
            if (neigborDistance == - 1 || newDistance < neigborDistance) {
                // Обновить distance, путь в previous и добавить в очередь neighbor
            }
        }
    }

    return {}; // Возвращаем пустой путь, если путь не найден
}
```

+ эвристика

Что такое эвристика

ИТМО

Эвристика - некое расстояние от текущей вершины к цели

Haversine formula - формула хаверсина

$$d = 2\,R * arcsin\sqrt{sin^2\left(\frac{\varphi_2 - \varphi_1}{2}\right) + cos\varphi_1 * cos\varphi_2 * sin^2\left(\frac{\lambda_2 - \lambda_1}{2}\right)}$$

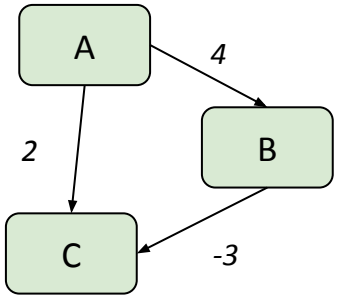
Евклидово расстояние в N-мерном пространстве

$$d_{Eucl}(p,q) =$$
$$= \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2 + \ldots + (q_n - p_n)^2} =$$
$$= \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}$$

Минутная пауза на осознание

Дейкстра – проблема отрицательных весов

Нужно найти путь из **A** до **C**



Видите проблему?
Из-за чего эта проблема?

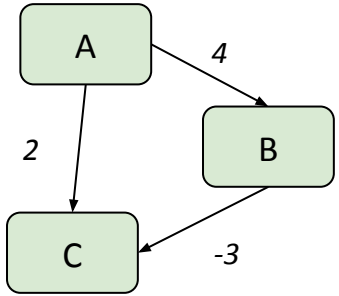
Алгоритм Беллмана — Форда

1. Создается матрица *dp* размером (1, V)

A	B	C
0	∞	∞

2. А дальше долгий нудный проход

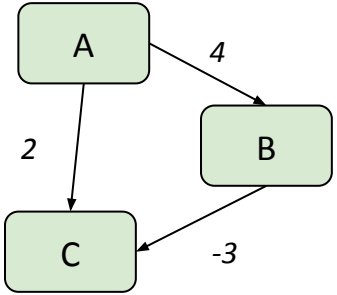
```
for v in V:
    for e in E:
        if dp[e.from] != ∞ & dp[e.from] + e.weight < dp[e.to]:
            df[e.to] = dp[e.from] + e.weight < dp[e.to]
```



3. Проверка на отрицательные циклы

Алгоритм Беллмана — Форда

```
for v in V:
   for e in E:
      if dp[e.from] + e.weight < dp[e.to]:
         df[e.to] = dp[e.from] + e.weight < dp[e.to]
```



Заполним таблицу

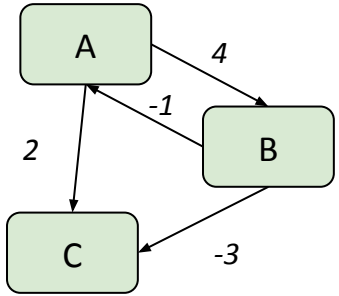
Итерация	A	B	C
0	0	∞	∞
1	0	4	2
2	0	4	**1**
3	0	4	**1**

Алгоритм Беллмана — Форда. Что там с циклами?

Итерация	A	B	C
0	0	∞	∞
1	-1	4	2
2	-1	3	**0**
3	-2	3	**0**

Сделаем еще один проход

Итерация	A	B	C
N	-2	2	-1



Если вес хоть где-то еще сильнее уменьшился, значит есть отрицательные циклы

Алгоритм Флойда Уоршелла

Алгоритм Флойда Уоршелла используется для нахождения кратчайших путей между всеми парами вершин в графе

Псевдокод (на основе матрицы смежности)

```
int dp[V, V], weight if exists or +∞
for i in V:
  dp_cur = copy(dp)  # копия dp с весами i
  for u in V:
    for v in V:
      if dp_cur_ui + dp_cur_iv < dp_cur_uv:
        dp_uv = dp_cur_ui + dp_cur_iv
```

, где i - промежуточная вершина

Мы еще поговорим об этом…

Алгоритм Флойда Уоршелла - добываем путь

Псевдокод (на основе матрицы смежности)

```
int dp[V, V], weight if exists or +∞
int path[V, V], ссылки на узлы

for i in V:
  dp_cur = copy(dp)  # копия dp с весами i
  for u in V:
    for v in V:
      if dp_cur_ui + dp_cur_iv < dp_cur_uv:
        dp_uv = dp_cur_ui + dp_cur_iv
        path_uv = path_ui
```

, где i - промежуточная вершина

Мне кажется

Или у вас есть вопросы?

ITsMOre than a UNIVERSITY

Ivan.Khodnenko@itmo.ru